

Day 45



IIFE - Immediately Invoked Function Expressions:

An IIFE is a function in JavaScript that runs immediately as soon as it is created. You don't need to call it separately. It executes instantly.

Why is it called "Immediately Invoked"?

Because:

- The function is written
- And right after, it is called automatically

In short:

Function is defined + executed at the same moment.

Syntax

The most common IIFE format is:

```
(function () {  
    console.log("I run immediately!");  
})();
```

Explanation:

- Function is wrapped inside () → makes it an expression
- Another () invokes it immediately

Where is IIFE used?

- To avoid global variables
- To create private variables
- To run setup code immediately
- To avoid variable conflicts in old JS (before modules)

Example: most basic example

```
JS main.js  
1 (function () {  
2   console.log("I run immediately!");  
3 })();  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS E:\JavaScript\Day41-50\Day45> node main.js  
I run immediately!
```

Destructuring assignment and Spread Operator:

What is Destructuring?

Destructuring allows you to extract values from arrays or objects and store them into variables easily.

It is just a short and clean way of writing:

- Take data from array/object
- Put into variables
- Without writing long code

Spread Operator (...)

The spread operator (...) is used to expand (spread) an array or object into individual elements. It spreads values out.

Difference Between Destructuring and Spread in Simple Words

Feature	Destructuring	Spread Operator
Purpose	Extract values	Expand values
Works On	Arrays, Objects	Arrays, Objects
Output	Variables	New array/object
Example	let {a} = obj	{...obj}

Example: array destructuring

```
JS destrucutirng.js > ...
1  const numbers = [1, 2, 3];
2  const [a, b, c] = numbers;
3  console.log(a); // 1
4  console.log(b); // 2
5  console.log(c); // 3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day41-50\Day45> node .\destrucutirng.js
1
2
3
```

Example:

```
JS destrucutirng.js > ...
1  const numbers = [1, 2, 3];
2  const [a, b, c] = numbers;
3  console.log(a,b,c);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day45> node .\destrucutirng.js
1 2 3
```

Example: printing something which is not there. It will give an error.

```
JS destrucutirng.js > ...
1  const numbers = [1, 2, 3];
2  const [a, b, c] = numbers;
3  console.log(a,b,c,d);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day41-50\Day45> node .\destrucutirng.js
E:\JavaScript\Day41-50\Day45\destrucutirng.js:3
● console.log(a,b,c,d);
```

Example: introducing "...rest"

```
JS destrucutirng.js > ...
1  const numbers = [1, 2, 3, 4, 6,7, 8];
2  let [a, b, c, ...rest] = numbers;
3  console.log(a,b,c, rest);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day45> node .\destrucutirng.js
1 2 3 [ 4, 6, 7, 8 ]
```

Example: skipping items.

```
5  let [x, , z] = [1, 2, 3];
6  console.log(x, z); // 1 3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day45> node .\destrucutirng.js
1 3
```

Example: default items.

```
8 let [a = 5, b = 10] = [];  
9 console.log(a, b); // 5 10
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day45> node .\destrucutirng.js  
5 10
```

Example: destructuring object

```
11 let user = { name: "Aditya", age: 23 };  
12 let { name, age } = user;  
13 console.log(name, age);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day45> node .\destrucutirng.js  
Aditya 23
```

Example: renaming variables

```
15 let { name: username } = user;  
16 console.log(username); // Aditya
```

Example: default values

```
17  
18 let { city = "Unknown" } = user;  
19 console.log(city); // Unknown
```

Example: spread operator

```
JS spread.js > ...  
1 let arr1 = [1, 2, 3];  
2 let arr2 = [...arr1];  
3  
4 console.log(arr2); // [1, 2, 3]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day41-50\Day45> node .\spread.js  
[ 1, 2, 3 ]
```

Example: merging array.

```
6   let a = [1, 2];
7   let b = [3, 4];
8   let c = [...a, ...b];
9
10  console.log(c); // [1, 2, 3, 4]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day41-50\Day45> node .\spread.js
[ 1, 2, 3, 4 ]
```

Example: spread with object

```
11  let user = { name: "Aditya", age: 23 };
12  let copy = { ...user };
13
14  console.log(copy); // same as user
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day41-50\Day45> node .\spread.js
{ name: 'Aditya', age: 23 }
```

--The End--